Text-to-SQL vérifié : interroger une base en français, avec contrôles explicites (octobre 2026)

Contact : salah.chadli@gmail.com

Prérequis : la séance « Bases de données » du cours

Description :

Poser une question en français à une base de données (« quels clients ont dépensé plus de 40 € au total ? ») est l'un des usages des LLM les plus demandés en entreprise. La difficulté vient de ce qu'un LLM produit du SQL plausible, qui s'exécute souvent sans erreur et renvoie pourtant un résultat faux (jointure oubliée, COUNT sur la mauvaise colonne, valeurs NULL ignorées). Il peut aussi produire une requête destructrice.

Dans ce projet, le LLM se contente de traduire. Votre système analyse la requête produite avec un parser écrit à la main et la vérifie avant de l'exécuter (tables et colonnes existantes, jointures sur des clés, agrégats cohérents avec le GROUP BY, lecture seule, limite de lignes). Il renvoie les erreurs au LLM pour correction, puis explique en français ce que fait la requête, à partir de son arbre syntaxique (AST). Ces contrôles ne prouvent pas que la requête répond à l'intention de l'utilisateur.

La base de départ est Chinook [1] (un magasin de musique en ligne : clients, factures, morceaux, artistes, une dizaine de tables). La fiabilité du système est mesurée sur un banc de questions que vous générez vous-mêmes, et dont la requête de référence est connue. Les formulations en français doivent aussi être vérifiées.

Ce que vous allez apprendre

- Sémantique fine de SQL : jointures, GROUP BY / HAVING, NULL, agrégats, multiensembles [2].
- Analyse syntaxique par descente récursive et arbre syntaxique (AST) [3], puis analyse sémantique (résolution des noms et des alias, types).
- Évaluation par exécution : deux requêtes différentes peuvent être correctes toutes les deux, c'est pourquoi on compare leurs résultats.
- Sécurité : exécution en lecture seule (connexion mode=ro, PRAGMA query_only, set_authorizer de sqlite3 [5]), requêtes malveillantes cachées dans la question.

Travail demandé : socle (obligatoire)

- Une grammaire et un parser manuels produisant un AST : SELECT, au plus un INNER JOIN sur clé déclarée, WHERE avec comparaisons reliées par AND, COUNT/SUM, GROUP BY, HAVING simple, ORDER BY, LIMIT. Définir types et NULL. Consommer toute l'entrée et rejeter le SQL hors sous-ensemble, notamment les sous-requêtes.
- Un vérificateur sur l'AST (Visitor), avec erreurs exploitables et deux appels LLM au plus (proposition initiale et une correction). Régénérer le SQL depuis l'AST validé, avec valeurs liées par paramètres et identifiants autorisés.
- Un générateur d'explication en français à partir de l'AST, déterministe (sans LLM).
- Au moins 200 questions générées. Exécution en lecture seule, fonctions autorisées, plafond de lignes et délai contrôlé par sqlite3. Un backend FastAPI et un front web : question, requête et erreurs surlignées, résultat, historique.

Extensions possibles, selon le temps disponible

- Plusieurs requêtes candidates et vote par résultat d'exécution, en mesurant ses erreurs : un accord ne garantit pas la bonne réponse.
- Détection d'ambiguïté (« les meilleurs clients » : par nombre d'achats ou par montant ?) et question de clarification.
- Sélection des tables pertinentes avant l'appel au LLM (schema linking), avec mesure de son effet.
- Seconde base issue de l'open data française, ou passage à PostgreSQL avec un rôle en lecture seule.

Évaluation chiffrée

Générer des requêtes de référence par gabarits sur Chinook, puis leurs questions. Vérifier que les reformulations conservent le sens. Comparer les multiensembles de lignes sans ordre imposé, et les séquences avec ORDER BY. Pour LIMIT, exiger un ordre total déterministe. Ajouter de petites bases avec cas limites pour limiter les accords accidentels. Mesurer exactitude, gain après réparation, blocage des requêtes malveillantes et acceptation des requêtes légitimes. BIRD [4] reste une lecture, pas le banc principal.

Stack suggérée

Python, FastAPI, Pydantic, SQLite (sqlite3), pytest + Hypothesis, HTML/JS, Ollama (Qwen2.5-Coder 1.5B Q4_K_M), GitHub Actions.

Règles du sujet

- Interdits dans le cœur : agents SQL de LangChain, Vanna, LlamaIndex et équivalents, ainsi que tout parser SQL existant. sqlglot est autorisé uniquement comme oracle dans les tests.
- Le LLM est appelé uniquement à travers une interface que vous définissez (LLMClient), avec au moins une implémentation réelle (Ollama en local) et une implémentation factice (FakeLLM) qui renvoie des réponses scriptées. Tous les tests et la CI tournent avec FakeLLM, sans réseau ni clé d'API.
- Modèle local conseillé : qwen2.5-coder:1.5b-instruct-q4_K_M via Ollama [M] (Q4_K_M : poids quantifiés sur environ 4 bits, fichier de 0,99 Go, distinct de la RAM requise). Machine visée : portable avec 8 Go de RAM totale, CPU Intel Core i5 / AMD Ryzen 5 récent (4 cœurs ou plus), ou Mac Apple M1 et suivants avec 8 Go de mémoire unifiée. Aucun GPU dédié requis. Sur CPU, les campagnes peuvent être lentes.
- Repères de dimensionnement, à vérifier sur la machine : avec 16 Go, essayer qwen2.5-coder:7b en Q4_K_M. Le 7B est facultatif. Commencer avec num_ctx=2048 (entrée et sortie), une seule requête et un seul modèle chargé. Réserver 5 Go de disque pour le petit modèle et les outils (10 Go pour le 7B). Fermer les applications lourdes. Sous 8 Go, commencer avec FakeLLM. Mesurer RAM et latence avant une campagne réelle. Téléchargement initial nécessaire, puis exécution locale sans API payante. Limiter le schéma fourni aux tables utiles. Mesurer les erreurs du petit modèle.

[1] https://github.com/lerocha/chinook-database

[2] https://www.sqlite.org/lang_select.html

[3] R. Nystrom, Crafting Interpreters, https://craftinginterpreters.com (chapitres 4 à 6)

[4] https://bird-bench.github.io

[5] https://docs.python.org/3/library/sqlite3.html

[O] https://ollama.com (sorties structurées : https://ollama.com/blog/structured-outputs)

[M] https://ollama.com/library/qwen2.5-coder:1.5b-instruct-q4_K_M

Verified text-to-SQL: querying a database in plain language, with explicit checks (October 2026)

Contact: salah.chadli@gmail.com

Prerequisites: the "Databases" session of the course

Description:

Asking a database a question in plain language ("which customers spent more than €40 in total?") is one of the most requested LLM use cases in industry. The problem: an LLM produces plausible SQL, which often runs without error and yet returns a wrong result (forgotten join, COUNT on the wrong column, ignored NULL values), and it may also produce a destructive query.

In this project, the LLM only translates. Your system parses the generated query with a hand-written parser, checks it before running it (existing tables and columns, joins on keys, aggregates consistent with the GROUP BY, read-only, row limit), sends errors back to the LLM for correction, then explains in plain language what the query does, from its abstract syntax tree (AST). These checks do not prove that the query matches the user's intent.

The starting database is Chinook [1] (an online music store: customers, invoices, tracks, artists, about ten tables). The system's reliability is measured on a bench of questions you generate yourselves, whose reference query is known. The wording of each question must also be checked.

What you will learn

- Fine-grained SQL semantics: joins, GROUP BY / HAVING, NULL, aggregates, multisets [2].
- Recursive-descent parsing and abstract syntax trees (AST) [3]. Semantic analysis (name and alias resolution, types).
- Execution-based evaluation: two different queries may both be correct, so their results are compared.
- Security: read-only execution (mode=ro connection, PRAGMA query_only, sqlite3's set_authorizer [5]), malicious requests hidden in the question.

Required work: core (mandatory)

- A hand-written grammar and parser producing an AST: SELECT, at most one INNER JOIN on a declared key, WHERE comparisons joined by AND, COUNT/SUM, GROUP BY, simple HAVING, ORDER BY, LIMIT. Define types and NULL. Consume the whole input and reject unsupported SQL, including subqueries.
- An AST checker (Visitor) with actionable errors and at most two LLM calls (initial proposal and one repair). Regenerate SQL from the validated AST, with bound value parameters and allowlisted identifiers.
- A deterministic plain-language explanation generator from the AST (no LLM).
- At least 200 generated questions. Read-only execution, allowlisted functions, a row cap and a deadline enforced through sqlite3. A FastAPI backend and web front-end: question, query and highlighted errors, result, history.

Possible extensions, depending on available time

- Several candidate queries and voting on execution results, measuring errors: agreement does not guarantee a correct answer.
- Ambiguity detection ("the best customers": by number of purchases or by amount?) and a clarification question.
- Selecting relevant tables before calling the LLM (schema linking), with a measurement of its effect.
- A second database from French open data, or a move to PostgreSQL with a read-only role.

Quantitative evaluation

Generate reference queries from Chinook templates, then questions. Check that paraphrases preserve meaning. Compare row multisets when order is unspecified, and sequences with ORDER BY. Require a deterministic total order for LIMIT. Add small databases with edge cases to reduce accidental agreement. Measure accuracy, repair gains, malicious-query blocking and legitimate-query acceptance. BIRD [4] remains reading material, not the main bench.

Suggested stack

Python, FastAPI, Pydantic, SQLite (sqlite3), pytest + Hypothesis, HTML/JS, Ollama (Qwen2.5-Coder 1.5B Q4_K_M), GitHub Actions.

Project rules

- Forbidden in the core: LangChain SQL agents, Vanna, LlamaIndex and the like, as well as any existing SQL parser. sqlglot is allowed only as an oracle in tests.
- The LLM is only called through an interface you define (LLMClient), with at least one real implementation (local Ollama) and one fake implementation (FakeLLM) returning scripted answers. All tests and the CI run with FakeLLM, with no network and no API key.
- Suggested local model: qwen2.5-coder:1.5b-instruct-q4_K_M through Ollama [M] (Q4_K_M: weights quantized to roughly 4 bits, 0.99 GB file, distinct from required RAM). Target machine: laptop with 8 GB total RAM, recent Intel Core i5 / AMD Ryzen 5 CPU (4 or more cores), or Apple M1 or newer Mac with 8 GB unified memory. No dedicated GPU required. CPU campaigns may be slow.
- Sizing guidelines, to validate on the machine: with 16 GB, try qwen2.5-coder:7b in Q4_K_M. 7B is optional. Start with num_ctx=2048 (input and output), one request and one loaded model. Reserve 5 GB of disk for the small model and tools (10 GB for 7B). Close memory-heavy apps. Below 8 GB, start with FakeLLM. Measure RAM and latency before a real campaign. Initial download required, then local execution without a paid API. Restrict the supplied schema to relevant tables. Measure small-model errors.

[1] https://github.com/lerocha/chinook-database

[2] https://www.sqlite.org/lang_select.html

[3] R. Nystrom, Crafting Interpreters, https://craftinginterpreters.com (chapters 4–6)

[4] https://bird-bench.github.io

[5] https://docs.python.org/3/library/sqlite3.html

[O] https://ollama.com (structured outputs: https://ollama.com/blog/structured-outputs)

[M] https://ollama.com/library/qwen2.5-coder:1.5b-instruct-q4_K_M